

Module 7: Lesson 3

Advanced topics in Deep Learning for Finance: Combinatorial Purged Cross-Validation



Outline

- ▶ Pitfalls of Walk Forward and K-fold Cross-Validation
- ▶ Combinatorial Purged Cross-Validation

Pitfalls of Walk Forward and K-fold Cross-Validation

In Lessons 1 and 2, we described and implemented the **walk forward** method to test the robustness of a DL-based trading strategy.

While the walk forward method is not wrong by itself, it suffers from some weaknesses:

- ▶ The walk forward only tests over a single scenario (the historical path), meaning that each observation only has one forecast.
- ▶ The results can be biased by the specific behavior of the data in the training sample.

Instead, we may opt for **cross-validation**:

- ▶ E.g., training a model for the S&P500 in the mostly-booming period 2010–2019 and testing its performance in the 2007–2008 crisis period.
- ▶ However, there may be leakage of information from the test to the training sample that may generate misleading results.
- ▶ We only get one forecast per observation, and we only obtain an interpretation of how a strategy would have performed under hypothetical paths.

Combinatorial Purged Cross-Validation

The Combinatorial Purged Cross-Validation (CPCV) method addresses the issue that WF and CV only test a single path.

With CPCV, we can generate a prespecified number φ of backtesting paths by generating different combinations of training/testing sets.

- ▶ While it purges the observations to avoid leakage.

Consider a time series of T observations (labels and input features) partitioned into N groups without shuffling.

- ▶ Each group is of size $\lfloor T/N \rfloor$, except for the last group, which is of size $T - \lfloor T/N \rfloor(N - 1)$.

Let $k \leq N/2$ denote the number of groups of observations that we want to use as a testing set from training a model with the remaining $N - k$ groups.

- ▶ The total number of tested groups is $k \binom{N}{N - k}$.

The total number of “paths” that we can backtest, given the choices of N and k , is equal to:

$$\varphi[N, k] = \frac{k}{N} \binom{N}{N - k}$$

Combinatorial Purged Cross-Validation: Train-Test splits

Let's consider splitting a dataset into $N = 4$ groups, G1, G2, G3, G4, out of which we want to form combinations of train-test splits where each test set includes $k = 2$ groups.

For each train/test split S1,..., S6, the table below reports, with "x"s, which groups belong to the test set.

	S1	S2	S3	S4	S5	S6
G1	x	x	x			
G2	x			x	x	
G3		x		x		x
G4			x		x	x

Combinatorial Purged Cross-Validation: Backtest paths

With the choice of $N = 4$ and $k = 2$, we can form $\varphi[4, 3] = 3$ backtest paths from the combinatorial method by concatenating the predictions in the different test samples.

	S1	S2	S3	S4	S5	S6
G1	1	2	3			
G2	1			2	3	
G3		1		2		3
G4			1		2	3

With these combinations, we can obtain a “distribution” of backtest statistics (say Sharpe ratios), instead of having a single estimate for each statistic.

- In contrast, in a standard k -fold CV or Walk Forward, we would obtain a single backtest path.

CPCV: Purging

The procedure above shares similarities with the K-fold Cross-Validation method, where some test sets precede the training data.

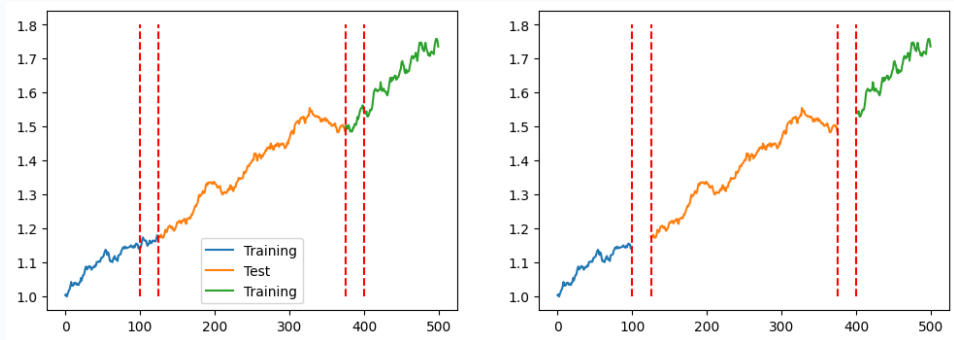
- ▶ This feature may lead to a leakage problem that can induce overfitting and that we must solve.
- ▶ The influence of leakage on the backtest statistics can be evaluated by taking $k \rightarrow T$.

Purging deals with those labels that overlap in their construction period.

- ▶ We must remove (purge) from the training set any observation whose label is based on the information that is also used in the test set.
- ▶ For example, consider the prediction problem for the 25-day return of an asset using daily data. To avoid leakage, we must purge from the training set the observations that include information 25 days before (after) the beginning (end) of the test sample.

By purging, we should not observe a monotonic improvement in performance out-of-sample as $k \rightarrow T$.

CPCV: Purging



CPCV: Embargoing

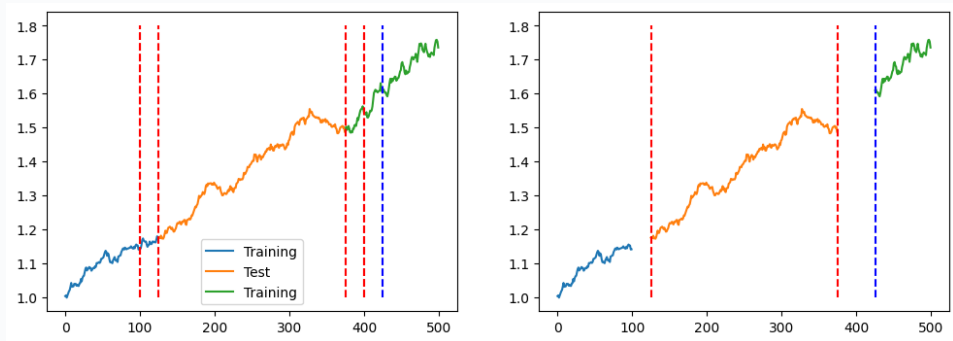
Purging may not prevent all leakage from the test to the training set due to the likely intrinsic serial correlation of the dataset:

- ▶ This problem only arises when the training set trails the test set.
- ▶ Information at the end of the test set may be serially correlated with that appearing in training information that goes after the test set.
- ▶ This correlation is a likely source of backtest overfitting.

We can **embargo** those training observations that are relatively close to the end of the test sample.

By embargoing, we should not observe a monotonic improvement in performance out of sample as $k \rightarrow T$.

CPCV: Purging and embargoing



CPCV: The algorithm

The full CPCV backtesting algorithm can be described as follows:

1. Partition the T observations of labels and input features into N groups without reshuffling. Groups $n = 1, \dots, N - 1$ are of size $\lfloor N/T \rfloor$ and group N is of size $T - \lfloor N/T \rfloor(N - 1)$.
2. Compute all the combinations of training/test splits. For each split, $N - k$ groups represent the training set, and k groups represent the test set.
3. For each train/test split, purge all the observations from the training sets whose labels span the period of the test sets. Apply an embargo to training observations that are preceded by the test set.
4. Train your model separately for all the combinations of training sets and compute predictions for their corresponding test sets.
5. Build the $\varphi[N, k]$ backtesting paths and obtain the distribution of backtest statistics of your choice.

For a detailed Python implementation, please check the references for this lesson.

Summary of Lesson 3

In Lesson 3, we have:

- Described the main ingredients of Combinatorial Purged Cross-Validation as a backtesting procedure.

⇒ **References for this lesson:**

López de Prado, M. “Advances in Financial Machine Learning: Lecture 4/10 (seminar slides).” *SSRN*. 29 September 2018.

Berend. “The Combinatorial Purged Cross-Validation method.” *Medium*, 31 March 2022.

TO DO NEXT: This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will describe how to introduce custom loss functions to train deep learning models in TensorFlow-Keras.

See you there!